

Phase 2 Test Specifications

POC-07 — Inventory Management & Procurement System

Total Test Cases: 20 | **Pass Threshold:** 14 of 20 (70%)

INGESTION TESTS (4 cases)

TC-07-P2-ING-01: Manual Loads

```
def test_manual_loads():
    from langchain_community.document_loaders import TextLoader
    import os
    assert os.path.exists("rag/inventory_manual.md")
    docs = TextLoader("rag/inventory_manual.md", encoding="utf-8").load()
    assert len(docs) > 0 and len(docs[0].page_content) > 100
```

TC-07-P2-ING-02: Chunks Within 600 Chars

```
def test_chunks_size():
    from langchain_community.document_loaders import TextLoader
    from langchain.text_splitter import RecursiveCharacterTextSplitter
    docs = TextLoader("rag/inventory_manual.md", encoding="utf-8").load()
    chunks = RecursiveCharacterTextSplitter(chunk_size=600,
    chunk_overlap=50).split_documents(docs)
    for c in chunks:
        assert len(c.page_content) <= 600
```

TC-07-P2-ING-03: At Least 20 Chunks

```
def test_min_chunks():
    from langchain_community.document_loaders import TextLoader
    from langchain.text_splitter import RecursiveCharacterTextSplitter
    docs = TextLoader("rag/inventory_manual.md", encoding="utf-8").load()
    chunks = RecursiveCharacterTextSplitter(chunk_size=600,
    chunk_overlap=50).split_documents(docs)
    assert len(chunks) >= 20
```

TC-07-P2-ING-04: ChromaDB Collection Exists

```
def test_chromadb_collection():
    import chromadb
```

```
client = chromadb.PersistentClient(path="./chroma_db")
assert "inventory_manual" in [c.name for c in client.list_collections()]
assert client.get_collection("inventory_manual").count() >= 20
```

RETRIEVAL TESTS (6 cases)

TC-07-P2-RET-01: SKU Format Query Returns Correct Pattern

```
def test_sku_query():
    from rag.rag_chain import build_rag_chain, ask_question
    result = ask_question("What is the SKU format for grocery products?",
        build_rag_chain())
    answer = result.get("answer", "").lower()
    assert any(x in answer for x in ["gro", "sku", "grocery", "prefix"])
```

TC-07-P2-RET-02: Top-K Returns 4

```
def test_top_k():
    from rag.rag_chain import build_rag_chain
    chain = build_rag_chain()
    retriever = chain.retriever if hasattr(chain, 'retriever') else chain
    if hasattr(retriever, 'search_kwargs'):
        assert retriever.search_kwargs.get('k', 4) == 4
```

TC-07-P2-RET-03: Irrelevant Query Low Score

```
def test_irrelevant_low_score():
    from unittest.mock import MagicMock
    mock = MagicMock()
    mock.query.return_value = {"documents": ["Inventory stock info."],
        "distances": [[0.87]]}
    assert mock.query(query_texts=["Football match results"], n_results=4)
        ["distances"][0][0] > 0.5
```

TC-07-P2-RET-04: PO Lifecycle Query

```
def test_po_lifecycle():
    from rag.rag_chain import build_rag_chain, ask_question
    result = ask_question("What are the stages of a purchase order?",
        build_rag_chain())
    answer = result.get("answer", "").lower()
```

```
assert any(w in answer for w in ["draft", "submitted", "received", "status",  
"lifecycle"])
```

TC-07-P2-RET-05: Empty Query Handled

```
def test_empty_query():  
    from rag.rag_chain import build_rag_chain, ask_question  
    try:  
        result = ask_question("", build_rag_chain())  
        assert isinstance(result, dict)  
    except Exception as e:  
        pytest.fail(f"Empty query raised: {e}")
```

TC-07-P2-RET-06: Latency Under 5 Seconds

```
def test_latency():  
    import time  
    from rag.rag_chain import build_rag_chain, ask_question  
    chain = build_rag_chain()  
    start = time.time()  
    ask_question("What is a reorder point?", chain)  
    assert time.time() - start < 5.0
```

GENERATION TESTS (6 cases)

TC-07-P2-GEN-01: Reorder Point Formula Explained

```
def test_reorder_formula():  
    from rag.rag_chain import build_rag_chain, ask_question  
    result = ask_question("How do I calculate a reorder point?",  
build_rag_chain())  
    answer = result.get("answer", "").lower()  
    assert any(w in answer for w in ["lead time", "daily", "demand", "safety",  
"reorder"])
```

TC-07-P2-GEN-02: PO Approval Threshold Correct

```
def test_po_approval():  
    from rag.rag_chain import build_rag_chain, ask_question  
    result = ask_question("When does a PO need Store Manager approval?",  
build_rag_chain())
```

```
answer = result.get("answer", "")
assert "50,000" in answer or "50000" in answer or "₹" in answer
```

TC-07-P2-GEN-03: Stock Movement Types Listed

```
def test_movement_types():
    from rag.rag_chain import build_rag_chain, ask_question
    result = ask_question("What are the stock movement types?", build_rag_chain())
    answer = result.get("answer", "").lower()
    assert any(w in answer for w in ["receipt", "sale", "adjustment", "transfer",
    "return"])
```

TC-07-P2-GEN-04: Out-of-Scope Declined

```
def test_out_of_scope():
    from rag.rag_chain import build_rag_chain, ask_question
    result = ask_question("What is the weather forecast for Mumbai?",
    build_rag_chain())
    answer = result.get("answer", "").lower()
    assert any(p in answer for p in ["don't have", "not in", "no information",
    "cannot"])
```

TC-07-P2-GEN-05: Non-Empty Answers

```
def test_non_empty():
    from rag.rag_chain import build_rag_chain, ask_question
    for q in ["What is SKU?", "What is FIFO?", "What is a stockout?"]:
        assert len(ask_question(q, build_rag_chain()).get("answer", "")) > 10
```

TC-07-P2-GEN-06: Category Management Differences

```
def test_category_management():
    from rag.rag_chain import build_rag_chain, ask_question
    result = ask_question("How do grocery products differ from electronics in
    inventory management?", build_rag_chain())
    answer = result.get("answer", "").lower()
    assert any(w in answer for w in ["grocery", "electronic", "shelf life",
    "velocity", "cost"])
```

OBSERVABILITY TESTS (4 cases)

TC-07-P2-OBS-01: LangSmith Trace

```
def test_langsmith():
    import os
    if not os.getenv("LANGCHAIN_API_KEY"): pytest.skip("No key")
    from langsmith import Client
    from rag.rag_chain import build_rag_chain, ask_question
    ask_question("What is a reorder point?", build_rag_chain())
    runs = list(Client().list_runs(project_name="AI-Readiness-POC-07-P2",
    limit=5))
    assert len(runs) > 0
```

TC-07-P2-OBS-02: OTel Spans

```
def test_otel_spans():
    from opentelemetry.sdk.trace import TracerProvider
    from opentelemetry.sdk.trace.export.in_memory_span_exporter import
InMemorySpanExporter
    from opentelemetry.sdk.trace.export import SimpleSpanProcessor
    from opentelemetry import trace
    exporter = InMemorySpanExporter()
    provider = TracerProvider()
    provider.add_span_processor(SimpleSpanProcessor(exporter))
    trace.set_tracer_provider(provider)
    from rag.rag_chain import build_rag_chain, ask_question
    ask_question("What is EOQ?", build_rag_chain())
    spans = exporter.get_finished_spans()
    assert any("rag" in s.name.lower() or "retrieve" in s.name.lower() for s in
spans)
```

TC-07-P2-OBS-03: Log Contains poc_id

```
def test_log_poc_id(capfd):
    from rag.rag_chain import build_rag_chain, ask_question
    ask_question("What is a stock movement?", build_rag_chain())
    out = capfd.readouterr().out + capfd.readouterr().err
    assert "POC-07" in out or True
```

TC-07-P2-OBS-04: Sources Returned

```
def test_sources():
    from rag.rag_chain import build_rag_chain, ask_question
    result = ask_question("How does PO receiving update stock?",
    build_rag_chain())
    assert isinstance(result, dict) and "answer" in result
```

```
assert any(k in result for k in ["source_documents", "sources"]) or \
    len(result.get("answer", "")) > 0
```